

AI-Native Fraud Defense System

A multi-agent risk desk for the AI era of fraud

Razorpay AI Buildathon | Track: AI Risk Manager | Prepared by Prateek

A multi-agent risk system that catches the fraud patterns classic detectors miss — transaction spikes and coordinated fake-account rings (the known problems), plus two emerging attack surfaces: AI shopping agents transacting outside their bounds, and AI-generated fake KYC documents, reviews, and dispute claims. Each agent produces an evidence-backed case file, not a black-box score, and a shared arbitrator resolves disagreements into a final block / allow / escalate decision with a full audit trail.

1. Problem Statement

Fraud detection today assumes the attacker is a human clicking through a checkout flow, and that identity documents, reviews, and dispute claims are written by real people. Both assumptions are breaking down:

- **Agentic commerce is arriving.** AI shopping agents will soon transact on a user's behalf. An agent using a stolen card, exceeding its owner's intent, or being manipulated by a merchant's checkout logic is a genuinely new attack surface with no mature defense today.
- **Generative AI is industrializing fraud content.** Fake KYC documents, fake reviews, and fake dispute narratives ('my order never arrived') can now be mass-produced by LLMs, cheaply and convincingly, at a scale manual review cannot keep up with.
- **Existing fraud systems — Razorpay's included — were built for human-committed fraud.** They are not built to ask "was this even a human?" or "was this document or text AI-generated?" That question is the actual gap this project addresses.

Core question the system answers: Was this transaction, document, or dispute produced by a human acting in good faith, an AI agent, or an AI-assisted fraud attempt — and how confident are we?

2. Why This, Why Now

- The **AI Risk Manager** track is explicitly built to surface risk- and ML-minded builders — success is measured on precision/recall and false-positive cost, not demo polish.
- A plain fraud classifier on a Kaggle dataset is the most common ML tutorial project that exists. The AI-native angle (agentic checkout abuse, synthetic content forensics) is a real, currently under-solved problem — not a rehash.
- This mirrors Razorpay's own stated direction: AI-enabled fraud is an emerging risk to Indian BFSI, and returns / chargebacks already quietly erode merchant margin.
- It plays directly to a cybersecurity + AI/ML background — adversarial thinking, evidence-based reasoning, and explainability over a single opaque model score.

3. Scope, Honestly

Razorpay already operates a mature, production-scale fraud system — this project will not out-build that in a few days, and it should not claim to. What is being demonstrated instead is problem understanding, sound architecture, honest evaluation, and a genuinely underexplored angle (AI-native fraud) rather than a rehash of a solved classification problem. The system also runs in **near real-time** (a replay pipeline processing transactions in sequence, latency measured per component) rather than claiming production-grade streaming infrastructure it does not have.

4. Architecture

One shared spine — an event bus and a common Case schema (evidence, confidence, cost basis, decision, audit log) — feeding a single dashboard. Five specialist agents write into it, each catching a distinct loss class.

Agent	Catches	Approach
1. Spike Sentinel	Bulk / bot fraud — card testing, rapid-fire abuse	Rolling-window velocity checks + gradient-boosted scorer (XGBoost) on transaction features
2. Ring Detector	Coordinated fake-account rings, including reused synthetic identities	Transaction graph over shared device/IP/bank-account edges + community detection (Louvain), LLM-verified against false positives like legitimate marketplaces
3. Agentic Checkout Guard	AI shopping agents acting outside their authorized bounds	Behavioral fingerprinting on session timing/cadence/entropy + a bounded policy engine (spend limits, step-up triggers)
4. Synthetic Content Forensics	AI-generated KYC docs, reviews, and dispute narratives	Linguistic fingerprinting (perplexity/burstiness, template reuse) + basic image forensics on ID documents
5. Adjudicator-in-Chief	Cross-agent disagreement on the same account/case	Pulls all relevant case files, weighs evidence, resolves or escalates to a human with both arguments shown

Agents 1 and 2 address the established loss classes; Agents 3 and 4 are the differentiators aimed at AI-native fraud specifically. Agent 5 is what turns this from a set of scripts into a system — it is the moment two agents corroborate or contradict each other that best demonstrates real reasoning, not just scoring.

5. Data

- **Base transactions:** Kaggle *kartik2112/fraud-detection* (Sparkov-simulated) — readable, non-PCA-obfuscated columns: amount, merchant, category, city, state, job, timestamp, fraud label.
- **Synthetic enrichment:** device ID, IP address, session velocity, merchant risk score — generated to mirror real production signals not present in the base dataset.
- **Synthetic content dataset:** a self-generated set of human-written vs. LLM-generated KYC descriptions, reviews, and dispute narratives, built specifically to train and evaluate Agent 4.
- **Split:** time-based train/held-out split (not random) to avoid leaking future spike patterns into training — keeps reported metrics defensible under scrutiny.

6. Evaluation Plan

- Precision, recall, and F1 per agent on held-out data — reported individually, not averaged away.
- A false-positive cost table shared across agents (a wrongful block costs differently depending on which agent triggered it).
- At least one cross-agent disagreement case walked through end-to-end in the demo.
- One deliberately handled failure case — a false positive escalated to a human instead of wrongly auto-blocked.
- Latency per pipeline stage, with an honest note on what a production streaming architecture would require.

7. Tech Stack

Layer	Tooling
Detection / ML	Python, pandas, scikit-learn, XGBoost, NetworkX + Louvain for graph clustering
Reasoning agents	Claude API for investigator/forensics/adjudicator agents, structured JSON case-file output
Service layer	FastAPI wrapping the pipeline; SQLite for the audit log
Dashboard	Streamlit or a lightweight React frontend, showing live case flow and the cost table
Stretch	Wire the transaction feed through Razorpay's test-mode sandbox so the demo looks API-driven, not a static CSV

8. Build Order

Sequenced so that a partial build is still a strong, complete submission at every stopping point.

- 1 **Spine + Case schema** — the foundation every agent plugs into.
- 2 **Spike Sentinel** — fastest to get working end-to-end; proves the pipeline.
- 3 **Synthetic Content Forensics** — the most novel, least-solved piece; built early so the differentiator is working even if time runs short.
- 4 **Ring Detector** — graph layer, additive on top of the existing pipeline.
- 5 **Agentic Checkout Guard** — hardest to source real behavioral data for; can be scoped down to a rules-based fingerprint if time is tight.
- 6 **Adjudicator-in-Chief** — only once at least two agents are solid and can genuinely disagree.

If time runs out after step 3, the result is already a working, genuinely novel two-agent system — a complete, honest submission, not a half-built five-agent mess.

9. Deliverables Mapped to Buildathon Requirements

Requirement	What it maps to
Public GitHub repo	Clean repo structure, README with architecture diagram, honest metrics reported as-is
5-minute pitch	Problem → live demo of a spike/synthetic-content flag → cross-agent disagreement → metrics → one handled failure case
Architecture writeup	The five-agent design above, why it beats a single classifier, and the shared cost-table reasoning
Panel interview	Defensible precision/recall numbers, a named scope boundary (not claiming to out-build Razorpay's production system), and a clear grasp of the AI-native fraud angle

Razorpay AI Buildathon — AI Risk Manager Track — Application closes 5 September